Université Paris Cité

Deep Learning

Année universitaire 2025–2026

Comparaison d'AlexNet et de DHVT

sur plusieurs jeux de données de classification d'images

Rapport de projet

Étudiants :	Haobo YANG (22513870)
	Zhenyu XU (22508276)
Cours :	Deep Learning
Remise :	April 1, 2026

Contents

1 Introduction **2**

2 Objectifs du projet **2**

3 Pourquoi comparer AlexNet et DHVT **2**

4 Jeu de données **3**

5 Modules essentiels de DHVT **4**
- 5.1 Module SOPE 4
- 5.2 Mécanisme d'attention 4
- 5.3 Module DAFF 5

6 Mise en oeuvre expérimentale **6**
- 6.1 Workflow général 6
- 6.2 Prétraitement 7
- 6.3 Entraînement 7

7 Résultats **8**
- 7.1 Comparaison principale entre CIFAR-10 et CIFAR-100 8
- 7.2 Comparaison globale entre les deux modèles 8
- 7.3 Différences par classe 10
- 7.4 Les images difficiles restent interprétables 11
- 7.5 Résultats sur les variantes de CIFAR-100 13

8 Discussion **13**

9 Conclusion **14**

10 Références **14**

1 Introduction

La classification d'images est l'une des tâches classiques de la vision par ordinateur. Un modèle reçoit une image en entrée et doit prédire la classe la plus probable parmi un ensemble de catégories fixées à l'avance. Dans ce projet, l'objectif n'est pas uniquement d'entraîner un modèle performant, mais de comparer deux familles d'architectures dans un cadre expérimental simple : les réseaux convolutionnels et les architectures de type transformer.

Le premier modèle étudié est AlexNet, un réseau convolutionnel important dans l'histoire du deep learning appliqué à la vision [1]. Le second est DHVT (*Dynamic Hybrid Vision Transformer*), une architecture proposée pour améliorer l'efficacité des Vision Transformers sur des jeux de données de petite taille [3]. Nous avons choisi de partir de jeux de données standards puis de construire des variantes visuelles plus ciblées, afin d'observer plus clairement les différences entre les deux approches.

2 Objectifs du projet

Ce projet poursuit trois objectifs. Le premier est de comparer AlexNet et DHVT sur plusieurs jeux de données de classification d'images, afin d'observer comment ces deux approches se comportent lorsque la difficulté augmente.

Le deuxième objectif consiste à concevoir ou modifier des jeux de données de manière à faire apparaître des différences de comportement entre CNN et transformers. Nous n'avons donc pas seulement utilisé des benchmarks standards ; nous avons aussi construit un petit benchmark dérivé de CIFAR-100 avec plusieurs variantes visuelles.

Le troisième objectif est plus pratique. Nous voulions implémenter nous-mêmes le pipeline d'entraînement en PyTorch, et reprendre les idées principales de DHVT à partir de l'article [3], dans une version adaptée au cadre du projet.

3 Pourquoi comparer AlexNet et DHVT

Comparer AlexNet et DHVT est pertinent parce que les deux modèles ne traitent pas l'information visuelle de la même manière. AlexNet représente une approche convolutionnelle classique, qui s'appuie fortement sur les voisinages spatiaux et les motifs de texture. DHVT, au contraire, appartient à la famille des Vision Transformers, tout en y ajoutant des mécanismes destinés à mieux fonctionner lorsque les données sont moins abondantes [3].

Dans le cadre du projet, ViT sert surtout de point de repère pour situer DHVT [2, 3]. En pratique, nos expériences portent bien sur deux modèles : AlexNet et DHVT. L'intérêt de cette comparaison est donc de voir si une architecture de type transformer, renforcée par quelques mécanismes supplémentaires, peut rivaliser avec un CNN sur de petits jeux de données et sur des variantes visuelles construites pour le test.

Le tableau 1 résume les différences structurelles les plus utiles pour situer les approches avant l'analyse expérimentale.

À quel niveau ?	AlexNet	ViT	DHVT
Logique générale	CNN classique.	Transformer de référence.	Transformer hybride pensé pour petit dataset.
Entrée	Convolutions locales.	Patch embedding direct.	SOPE ajoute une entrée plus locale et recouvrante.
Attention	Pas d'attention explicite.	MHSA standard.	HI-MHSA ajoute des head tokens pour structurer l'agrégation globale.
Bloc interne	Blocs convolutionnels.	MLP standard.	DAFF ajoute depthwise convolution et recalibrage du class token.
Idée principale	Modèle simple à lire.	Modèle global mais plus exigeant.	Corriger certaines limites de ViT sans revenir à un CNN pur.

Table 1: Repères structurels entre AlexNet, ViT et DHVT.

4 Jeu de données

Nous avons utilisé trois cadres expérimentaux. Les deux premiers reposent sur des jeux de données standards, CIFAR-10 et CIFAR-100. Ils servent à établir une comparaison de base entre les deux architectures, d'abord sur une tâche simple, puis sur une classification plus fine.

Le troisième cadre repose sur un petit benchmark construit à partir de CIFAR-100, en sélectionnant 12 classes (`bridge`, `bus`, `castle`, `leopard`, `orchid`, `poppy`, `road`, `rose`, `skyscraper`, `tiger`, `train`, `willow_tree`), avec 80 images d'entraînement et 30 images de validation par classe. Le sous-ensemble est construit avec une graine fixe afin de garder la même sélection pour tous les essais.

Sur ce sous-ensemble, nous avons défini quatre variantes :

- `clean` : sous-ensemble équilibré sans transformation supplémentaire ;
- `texture` : mélange local de patches afin d'affaiblir la structure globale tout en conservant une partie de l'information texturale ;
- `long_range` : réduction forte de la résolution puis remontée à la taille initiale, pour supprimer les détails fins et ne garder qu'une structure plus grossière ;
- `damaged_occluded` : masquage partiel et dégradation d'une zone de l'image.

L'idée n'est pas de proposer un benchmark général, mais de construire des conditions visuelles simples qui rendent les différences plus lisibles. L'objectif est de disposer d'un cadre où les réactions respectives des CNN et des architectures de type transformer puissent être observées plus directement.

Les principaux jeux de données utilisés dans les expériences finales sont résumés dans le tableau 2.

Option	Description
`CIFAR-10`	Jeu de données standard à 10 classes, utilisé comme base principale de comparaison.
`CIFAR-100`	Jeu de données à 100 classes, plus fin et plus difficile, avec le même format d'image.
`CIFAR-100 modifié`	Sous-ensemble contrôlé de CIFAR-100 avec variantes `clean`, `texture`, `damaged_occluded` et `long_range`.

Table 2: Principaux jeux de données utilisés dans les expériences finales.

Dans tous les cas, les deux modèles sont entraînés et évalués sur le même split pour un jeu de données donné. Cela permet de comparer les comportements observés sans changer simultanément les données et l'architecture.

5 Modules essentiels de DHVT

Cette partie présente les principaux modules de DHVT, utiles pour comprendre nos résultats [3]. Dans notre implémentation, nous avons repris les trois composants principaux : SOPE, HI-MHSA et DAFF.

5.1 Module SOPE

Le module SOPE (*Shared Overlapped Patch Embedding*) remplace le découpage direct de l'image en patches non recouvrants par une extraction progressive fondée sur des convolutions avec stride. L'idée est de conserver davantage d'information locale au moment de former les tokens envoyés au transformer.

On note l'image d'entrée

$$X \in \mathbb{R}^{B \times C \times H \times W},$$

où B désigne la taille du batch, C le nombre de canaux de l'image, H sa hauteur et W sa largeur. Après le passage dans SOPE, on obtient un tenseur

$$E \in \mathbb{R}^{B \times N \times d},$$

où N est le nombre total de tokens produits pour chaque image et d la dimension d'embedding de chaque token. Autrement dit, pour chaque image du batch, SOPE transforme une grille de pixels de taille $C \times H \times W$ en une séquence de N vecteurs de dimension d.

L'intérêt principal de SOPE est de faire intervenir davantage l'information locale dès l'entrée du modèle. Sur de petites images et avec peu de données, cela peut aider à préserver des détails utiles dès les premières couches.

5.2 Mécanisme d'attention

Dans un ViT standard, l'attention multi-têtes (*Multi-Head Self-Attention*, MHSA) part d'une séquence de tokens

$$Z \in \mathbb{R}^{B \times M \times d},$$

où B est la taille du batch, M le nombre total de tokens dans la séquence, et d la dimension d'embedding. À partir de cette séquence, on calcule les projections *query*, *key* et *value*, puis la

sortie s'écrit

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Dans cette expression, Q, K et V désignent les tenseurs de requêtes, de clés et de valeurs, et d_k la dimension des vecteurs de clé utilisée pour normaliser le produit scalaire $QK^\top$. Cette formulation permet de modéliser des relations globales entre les tokens de l'image, mais elle reste assez générique dans le cas d'un petit dataset.

Dans DHVT, ce mécanisme est modifié par l'ajout de *head tokens*, qui servent à structurer l'information globale avant l'attention. On considère une séquence d'entrée

$$Z = [x_{\text{cls}}; x_1; \ldots; x_N] \in \mathbb{R}^{B \times (N+1) \times d},$$

où x_{cls} est le *class token*, $x_1, \ldots, x_N$ les N patch tokens, et d la dimension d'embedding. La dimension d est répartie sur h têtes d'attention, et DHVT construit pour chacune d'elles un *head token* supplémentaire à partir de l'information moyenne contenue dans la séquence. Ces head tokens sont ensuite reprojetés dans l'espace de dimension d.

La séquence envoyée dans l'attention devient alors

$$\tilde{Z} = [x_{\text{cls}}; x_1; \ldots; x_N; \tilde{t}_1; \ldots; \tilde{t}_h].$$

Elle contient donc $N + 1 + h$ tokens : le class token, les N patch tokens, puis les h head tokens ajoutés.

L'attention est alors calculée sur cette séquence enrichie :

$$\text{HI-MHSA}(\tilde{Z}) = \text{softmax}\left(\frac{\tilde{Q}\tilde{K}^\top}{\sqrt{d_k}}\right) \tilde{V}.$$

Ici, $\tilde{Q}$, $\tilde{K}$ et $\tilde{V}$ désignent les projections calculées à partir de $\tilde{Z}$. Dans l'esprit du papier, ces *head tokens* servent à mieux faire interagir les groupes de canaux associés aux différentes têtes d'attention.

Après l'attention, la sortie des head tokens est réinjectée dans le class token par moyenne :

$$x^\star_{\text{cls}} = x'_{\text{cls}} + \frac{1}{h} \sum_{i=1}^{h} \tilde{t}'_i.$$

Ici, x'_{cls} désigne le class token après le calcul d'attention, $\tilde{t}'_i$ la sortie du i-ième head token après attention, et $x^\star_{\text{cls}}$ le class token final utilisé ensuite pour la classification. Cette étape montre que les head tokens ne servent pas seulement à enrichir l'attention : ils participent aussi directement à la construction de la représentation globale finale.

5.3 Module DAFF

Le module DAFF (*Dynamic Adaptive Feed-Forward*) joue le rôle d'une couche feed-forward enrichie. Dans l'implémentation, il comprend :

- une première projection linéaire ;
- une convolution depthwise ;
- une activation GELU ;

- une seconde projection linéaire ;
- un recalibrage de type channel-attention appliqué au class token.

On note T la matrice des patch tokens en entrée du bloc, c'est-à-dire l'ensemble des tokens spatiaux, à l'exclusion du class token. On peut résumer le bloc DAFF par :

$$T_1 = W_1 T,$$

$$T_2 = \mathrm{DWConv}(T_1),$$

$$T_3 = W_2 \,\mathrm{GELU}(T_2).$$

Dans ces trois étapes, W_1 et W_2 désignent deux projections linéaires apprises, et DWConv une convolution *depthwise* appliquée séparément par canal afin de réintroduire une interaction locale. On construit ensuite un vecteur de poids de canaux w appliqué au class token :

$$w = \sigma(f(\mathrm{mean}(T_3))),$$

$$x'_{\mathrm{cls}} = x_{\mathrm{cls}} \odot w,$$

où $\mathrm{mean}(T_3)$ désigne une agrégation moyenne sur les patch tokens, f une transformation apprise, σ la sigmoïde, et $\odot$ le produit élément par élément. Le vecteur w sert donc de mécanisme de recalibrage canal par canal du class token.

L'intérêt de DAFF est double : conserver une composante locale grâce à la convolution *depthwise*, et renforcer le *class token* à partir de l'information produite par les patch tokens. Ce module participe ainsi à l'idée générale de DHVT : rendre un transformer plus robuste dans le régime *small-data*.

6 Mise en oeuvre expérimentale

Cette partie présente les choix concrets utilisés dans les notebooks finaux. Nous avons cherché à garder un protocole simple et homogène : même split entraînement/validation, même budget d'entraînement, sauvegarde du meilleur checkpoint sur l'accuracy de validation, puis analyse comparative.

6.1 Workflow général

Le workflow global reste le même pour les deux modèles, comme indiqué dans le tableau 3. L'idée était de garder une procédure claire : préparer les données, entraîner les modèles, sauvegarder les meilleurs poids, puis analyser les résultats avec plusieurs visualisations.

Étape	Procédure
1	Choisir l'environnement d'exécution (local ou Colab).
2	Choisir le jeu de données.
3	Prétraitement des images.
4	Initialisation du modèle.
5	Entraînement du modèle.
6	Évaluation sur l'ensemble de validation.
7	Sauvegarde des meilleurs poids.
8	Rechargement des poids sauvegardés.
9	Résumé comparatif final selon le notebook considéré.

Table 3: Workflow général de l'expérience.

Pour que la comparaison reste cohérente, AlexNet et DHVT sont toujours entraînés sur le même jeu de données, avec le même nombre d'époques et les mêmes règles de sélection du meilleur modèle. En revanche, le prétraitement des images reste adapté à chaque architecture.

6.2 Prétraitement

Dans tous les notebooks, AlexNet reçoit des images redimensionnées en `224×224`, tandis que DHVT travaille en `32×32`. Ce choix ne signifie pas que les entrées sont strictement identiques, mais il permet d'utiliser chaque architecture dans un cadre compatible avec sa structure. Les deux branches utilisent une normalisation simple `(0.5, 0.5, 0.5)`. L'entraînement repose sur une augmentation modérée des données, alors que l'évaluation s'effectue avec des transformations fixes.

6.3 Entraînement

Tous les modèles sont entraînés depuis zéro avec AdamW pendant huit époques. Le scheduler utilisé est un *Cosine Annealing Learning Rate*. Les batch sizes sont fixés de manière simple pour rester compatibles avec les ressources disponibles. Le meilleur modèle est sélectionné selon l'accuracy de validation.

Les notebooks ont été exécutés sur différentes plateformes selon les ressources disponibles, mais ce point reste secondaire. L'essentiel est que les trois notebooks suivent le même principe : comparaison directe des deux architectures, sans préentraînement, avec un budget d'entraînement identique.

Notebook	Rôle dans le projet
`v3_CIFAR-10.ipynb`	Expérience de base, comparaison principale sur un jeu simple et analyse qualitative détaillée.
`v3_CIFAR-100.ipynb`	Même protocole sur un jeu plus difficile, pour tester la capacité de généralisation sur une classification plus fine.
`v4_CIFAR-100_Variants.ipynb`	Benchmark complémentaire sur plusieurs variantes (`clean`, `texture`, `damaged_occluded`, `long_range`) afin d'observer plus précisément les différences entre AlexNet et DHVT.

Table 4: Les trois notebooks remis et leur rôle dans l'analyse expérimentale.

7 Résultats

Dans les trois notebooks, DHVT obtient de meilleurs résultats qu'AlexNet sur les expériences réalisées. L'écart est déjà visible sur CIFAR-10, puis il devient plus net sur CIFAR-100. Sur la version modifiée de CIFAR-100, DHVT garde également un avantage sur l'ensemble des variantes testées.

7.1 Comparaison principale entre CIFAR-10 et CIFAR-100

Mesure	CIFAR-10	CIFAR-100
Accuracy AlexNet / DHVT	0.6848 / 0.7762	0.2856 / 0.5012
Écart d'accuracy	+0.0914	+0.2156
F1 macro AlexNet / DHVT	0.6814 / 0.7759	0.2713 / 0.4958
Paramètres AlexNet / DHVT	58.32M / 6.80M	58.69M / 6.81M
Temps AlexNet / DHVT	24.73 min / 15.56 min	15.60 min / 14.24 min

Table 5: Comparaison horizontale entre AlexNet et DHVT sur les deux jeux de données principaux.

Le tableau 5 résume le résultat principal du projet. Sur CIFAR-10, DHVT dépasse déjà AlexNet. Sur CIFAR-100, l'écart devient beaucoup plus grand : le gain d'accuracy passe d'environ 9 points à plus de 21 points. Dans nos expériences, l'écart devient donc surtout plus net lorsque la classification est plus fine.

Cette tendance est cohérente avec l'idée mise en avant dans l'article de DHVT [3] : une architecture de type transformer peut rester pertinente sur des jeux de données modestes lorsqu'elle intègre davantage d'information locale. Le fait que DHVT reste beaucoup plus compact qu'AlexNet apporte aussi un élément utile à la comparaison.

7.2 Comparaison globale entre les deux modèles

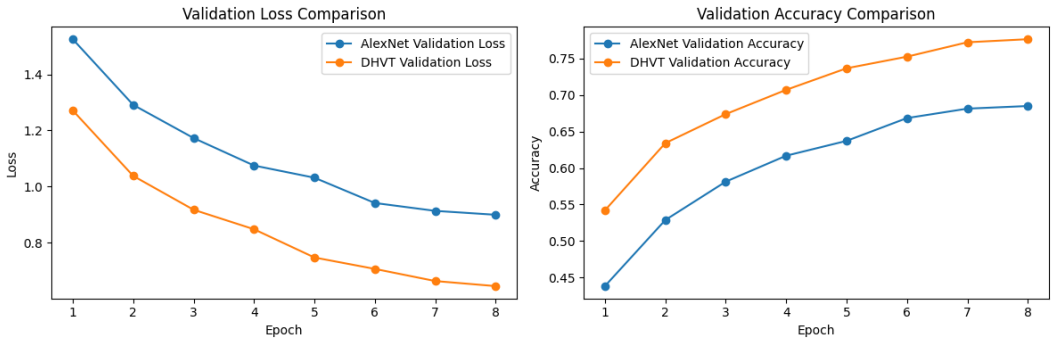

Figure 1: Comparaison DHVT-AlexNet sur CIFAR-10.

La figure 1 compare les courbes de validation sur CIFAR-10. Dès la deuxième époque, DHVT atteint environ 0.63 d'accuracy de validation, contre environ 0.53 pour AlexNet, et cet écart reste visible jusqu'à la fin. Du côté de la perte, la diminution est plus rapide et plus régulière pour DHVT : à l'époque 8, sa loss est proche de 0.64 alors que celle d'AlexNet reste autour de

0.90. La figure montre donc non seulement un meilleur score final, mais aussi une convergence plus rapide et une trajectoire plus stable.

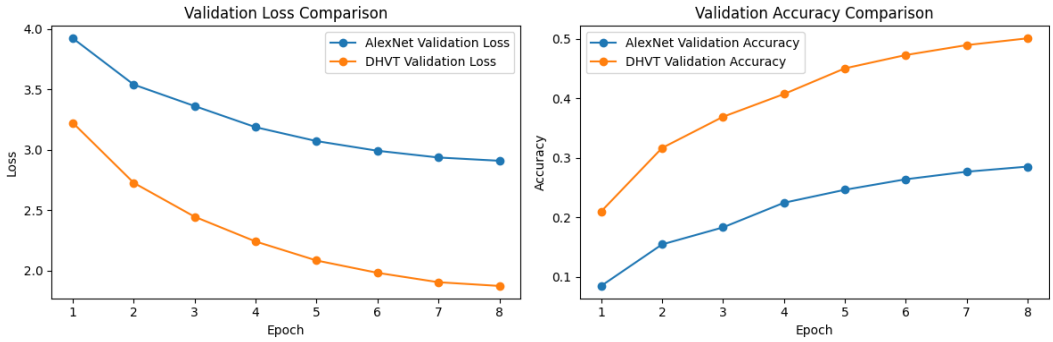

Figure 2: Comparaison DHVT-AlexNet sur CIFAR-100.

La figure 2 présente la même comparaison sur CIFAR-100. Ici, l'écart se creuse davantage : l'accuracy de DHVT passe d'environ 0.21 à 0.50, alors qu'AlexNet progresse seulement d'environ 0.08 à 0.29. La différence est tout aussi nette sur la perte de validation, qui descend sous 1.9 pour DHVT mais reste proche de 2.9 pour AlexNet en fin d'entraînement. La figure montre donc que, sur ce jeu de données, DHVT reste devant pendant toute la phase d'apprentissage.

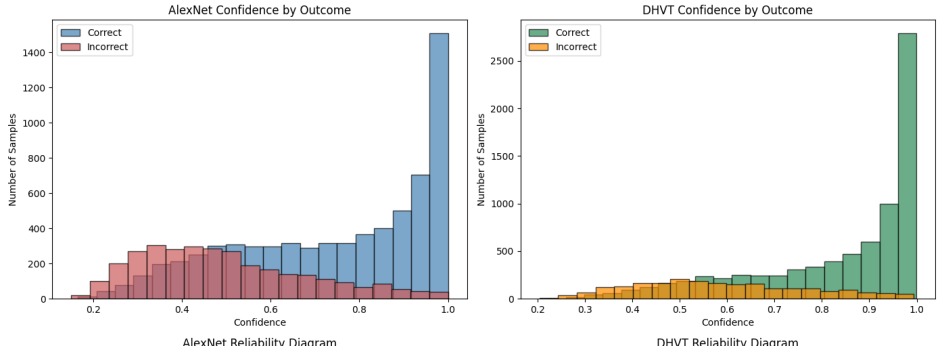

Figure 3: Distribution des scores de confiance sur CIFAR-10.

La figure 3 montre la distribution des scores de confiance sur CIFAR-10. Chez AlexNet, les prédictions incorrectes occupent encore fortement la zone intermédiaire autour de 0.3–0.6, ce qui crée un recouvrement important avec les prédictions correctes. Chez DHVT, au contraire, les prédictions correctes se concentrent beaucoup plus près de 0.9–1.0, tandis que les erreurs restent majoritairement dans des niveaux de confiance plus bas. La distribution indique donc une séparation plus nette entre succès et erreurs, et pas seulement une hausse uniforme des scores.

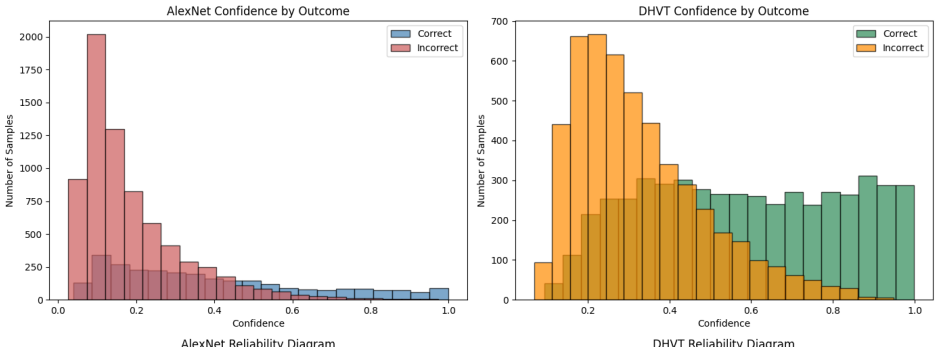

Figure 4: Distribution des scores de confiance sur CIFAR-100.

La figure 4 reprend cette lecture sur CIFAR-100. Pour AlexNet, une grande partie des prédictions, y compris correctes, reste concentrée à très faible confiance, souvent sous 0.3, ce qui traduit un modèle hésitant. DHVT présente encore un recouvrement entre correct et incorrect, mais ses bonnes prédictions s'étendent davantage vers les zones 0.5–1.0 et ses erreurs se concentrent surtout dans les niveaux plus bas. Même si la tâche est plus difficile, la structure de la distribution reste donc plus favorable pour DHVT.

7.3 Différences par classe

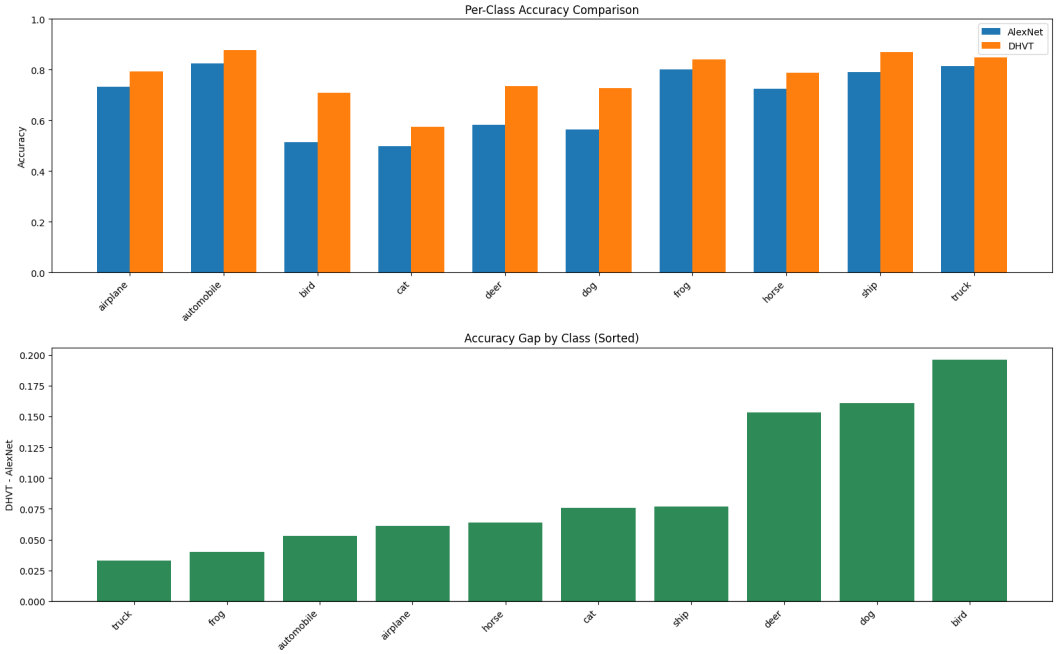

Figure 5: Accuracy par classe sur CIFAR-10.

La figure 5 montre que le gain de DHVT n'est pas concentré sur une seule classe. Les plus grands écarts apparaissent pour *bird*, *dog* et *deer*, avec des gains proches ou supérieurs à 0.15. À l'inverse, sur *truck*, *frog* ou *automobile*, l'avantage reste positif mais beaucoup plus modeste. La figure montre donc que l'amélioration observée n'est pas limitée à un petit nombre de classes.

7.4 Les images difficiles restent interprétables

Figure 6: Exemples d'erreurs de classification.

Les exemples de la figure 6 montrent que les erreurs restantes ne sont pas toutes de même nature. Dans plusieurs cas, DHVT corrige une confusion plausible d'AlexNet, par exemple *ship/automobile*, *cat/dog* ou *ship/frog*. On voit aussi qu'il ne résout pas tout : sur un bateau vu de très près, DHVT prédit *airplane*, et sur une voiture blanche sur fond rouge, il prédit *cat*. La figure montre donc à la fois des corrections intéressantes par rapport à AlexNet et des cas où l'ambiguïté visuelle reste forte.

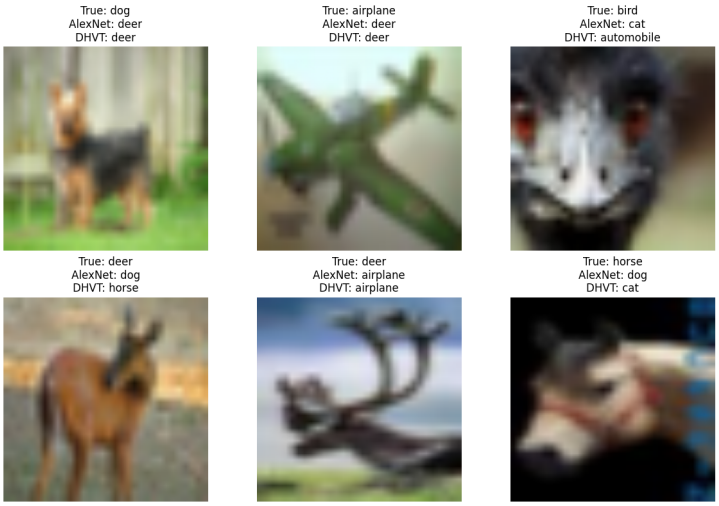

Figure 7: Exemples difficiles pour les deux modèles.

La figure 7 rassemble des cas où les deux modèles se trompent. Plusieurs exemples correspondent à des silhouettes très floues ou à des objets vus sous un angle inhabituel : un chien assimilé à un *deer*, un avion vert classé comme *deer*, ou encore un oiseau rapproché dont la texture sombre conduit à des prédictions incohérentes. Même lorsque les deux modèles se trompent différemment, la difficulté visuelle de l'image reste manifeste. Cette figure sert donc surtout à délimiter les cas où l'ambiguïté du dataset domine la différence architecturale.

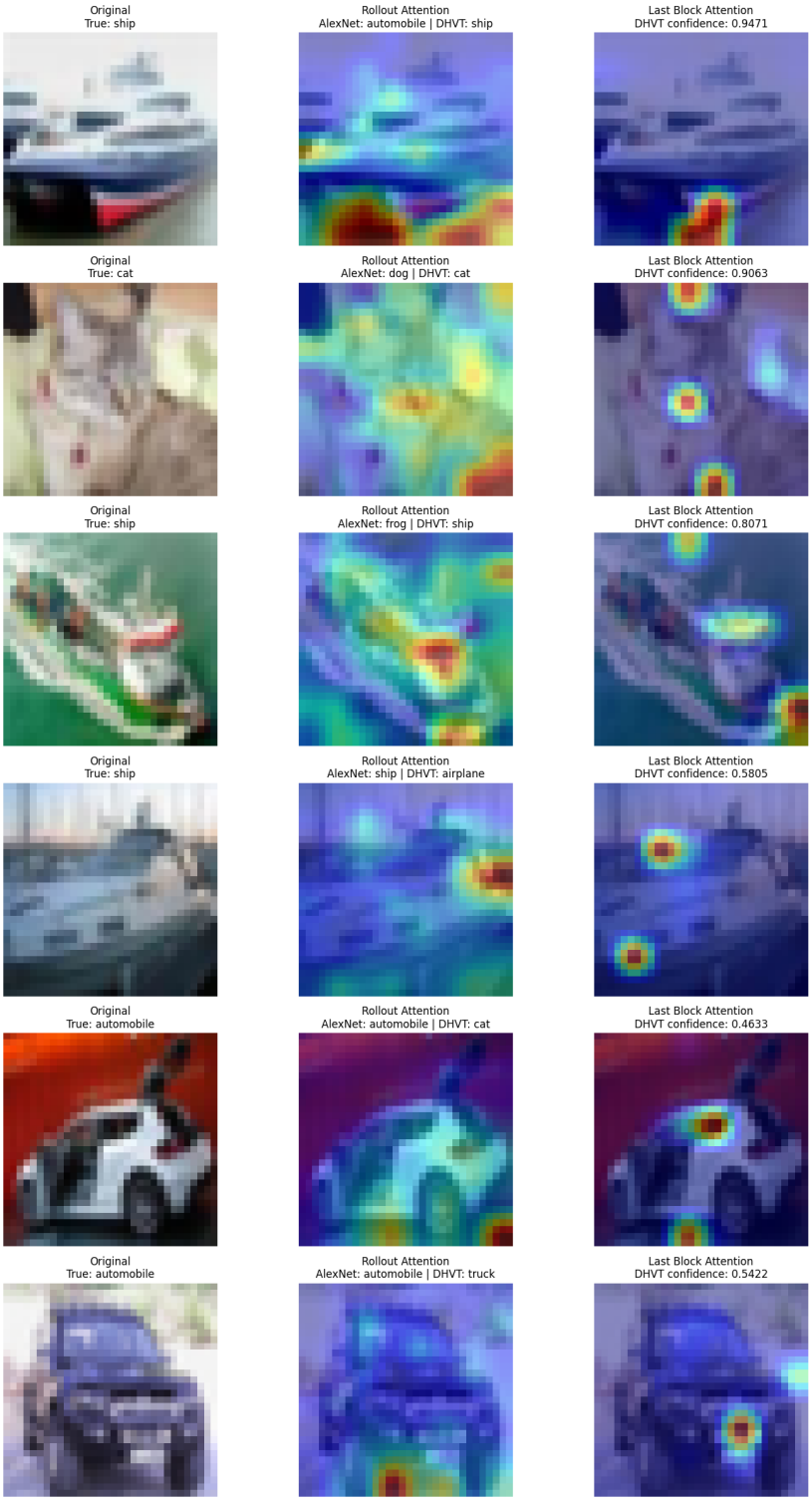

Figure 8: Cartes d'attention de DHVT sur des exemples de validation.

La figure 8 permet de relier certaines prédictions à des zones effectivement regardées par le modèle. Sur les exemples correctement classés, comme le bateau ou le chat, les cartes les plus intenses se concentrent sur des régions discriminantes de l'objet plutôt que sur l'ensemble de l'arrière-plan ; sur le bateau dans l'eau, l'activation se rassemble notamment sur la coque et la zone de contraste la plus marquée. En revanche, dans les exemples erronés, l'attention se fixe plus volontiers sur des points très localisés ou sur des zones qui ne suffisent pas à identifier la

classe, comme une petite partie de carrosserie ou un détail brillant. La figure ne démontre pas à elle seule la cause des erreurs, mais elle donne une lecture qualitative cohérente avec les succès et les échecs observés.

7.5 Résultats sur les variantes de CIFAR-100

Pour aller plus loin, un troisième notebook a construit une version plus petite et plus contrôlée de CIFAR-100, avec quatre variantes : `clean`, `texture`, `damaged_occluded` et `long_range`. Le benchmark repose sur 12 classes, avec 80 images d'entraînement et 30 images de validation par classe. L'idée était de voir si l'écart entre AlexNet et DHVT restait visible quand on modifie la nature de l'information visuelle.

Variante	AlexNet acc.	DHVT acc.	Écart DHVT – AlexNet
clean	0.2694	0.5167	+0.2472
texture	0.2583	0.4278	+0.1694
damaged_occluded	0.2944	0.4222	+0.1278
long_range	0.3333	0.4472	+0.1139

Table 6: Résultats du notebook modifié sur les variantes de CIFAR-100.

Le tableau 6 montre que DHVT reste au-dessus dans tous les cas. Le gain le plus fort apparaît sur la version `clean` (+0.2472), puis sur `texture` (+0.1694), `damaged_occluded` (+0.1278) et `long_range` (+0.1139). Autrement dit, même quand on réduit le jeu de données et qu'on introduit des difficultés plus ciblées, la hiérarchie observée entre les deux modèles ne s'inverse pas.

L'ampleur de l'écart n'est toutefois pas exactement la même selon la perturbation. Sur `texture`, l'avantage de DHVT reste important. Sur `long_range`, l'écart reste positif mais un peu plus réduit. Enfin, sur `damaged_occluded`, le gain est encore présent mais plus modeste. Ces différences montrent que les transformations n'affectent pas les deux modèles de la même manière, même si la hiérarchie générale reste la même.

8 Discussion

Le point principal est que la comparaison entre les deux modèles reste cohérente dans trois cadres différents : un jeu de données simple (CIFAR-10), un jeu de données plus difficile (CIFAR-100), puis une version modifiée et plus ciblée de CIFAR-100.

Ces résultats restent cohérents avec la motivation du papier DHVT [3]. Les modules *SOPE*, *HI-MHSA* et *DAFF* donnent une lecture possible des écarts observés, en combinant des mécanismes globaux issus des transformers et des traitements plus locaux inspirés des CNN.

Il faut toutefois rappeler qu'une architecture de type transformer garde aussi des limites. En général, ces modèles sont plus sensibles au protocole d'entraînement, à la quantité de données et aux choix d'optimisation qu'un CNN très classique. Ils demandent aussi des coûts de mise au point plus importants, aussi bien pour le réglage des hyperparamètres que pour l'analyse de leur comportement interne. Enfin, leur interprétation reste souvent moins immédiate que celle d'un baseline convolutionnel simple, notamment lorsque l'on cherche à relier une décision finale à des indices visuels précis.

Il faut bien sûr garder les limites du protocole en tête. Huit époques restent courtes, et les variantes de CIFAR-100 sont des constructions expérimentales destinées à faire ressortir

certains comportements, pas un benchmark universel. Dans ce cadre, les résultats obtenus restent favorables à DHVT sur l'ensemble des comparaisons effectuées.

9 Conclusion

Sur CIFAR-10, sur CIFAR-100, puis sur les variantes construites à partir de CIFAR-100, DHVT obtient dans nos expériences des résultats supérieurs à ceux d'AlexNet. L'écart est déjà visible sur le jeu de données simple, puis il devient plus marqué quand la tâche demande une discrimination plus fine ou quand les conditions visuelles sont modifiées.

Cette régularité montre surtout que, dans notre cadre expérimental, DHVT constitue une alternative crédible à AlexNet pour les petits jeux de données [3]. Le modèle se distingue notamment par la combinaison de mécanismes globaux issus des transformers et de traitements plus locaux inspirés des CNN.

En conclusion, dans les trois notebooks étudiés, DHVT obtient des résultats supérieurs à ceux d'AlexNet. La conclusion principale du projet porte donc sur cette comparaison directe entre un baseline convolutionnel classique et une architecture hybride inspirée des transformers, dans les limites des expériences réalisées.

10 Références

References

[1] Krizhevsky, A., Sutskever, I., and Hinton, G. E. *ImageNet Classification with Deep Convolutional Neural Networks*. 2012.

[2] Dosovitskiy, A. et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021.

[3] Lu, Y. et al. *Bridging the Gap Between Vision Transformers and Convolutional Neural Networks on Small Datasets*. 2022.

[4] PyTorch Documentation. https://pytorch.org/

[5] Torchvision Documentation. https://pytorch.org/vision/stable/